08 · Git 与团队协作

> 本节目标：从"会用 Git 提交"到"能在团队里优雅协作"——分支策略、PR 流程、Code Review、CI 自动化、Issue 管理一站搞定。
>
> 学完本节，你应当能独立用 Git 完成日常开发、发起 Pull Request、用 GitHub Actions 跑自动化流水线，并在团队仓库里高效沟通。
>
> 📚 系列目录见 README.md
>
> ⚠️ Git 是 **几乎所有现代软件工程** 的地基——不管你学 Vue / React / 后端 / 嵌入式，它都跑不掉。把这一节读透，能省下后面无数次"找代码丢了怎么办"的痛苦。

目录

- 1. Git 是什么？为什么必须学?
- 2. 安装与首次配置
- 3. 核心概念：三个区、四种状态
- 4. 基础流程：本地 + 远程
- 5. 分支策略：Trunk-Based / Git Flow / GitHub Flow
- 6. 团队协作：Pull Request 与 Code Review
- 7. 冲突处理：什么时候 rebase，什么时候 merge
- 8. 提交规范：Conventional Commits
- 9. 高频救命命令：找回误删 / 回退版本 / 改 commit 历史
- 10. `.gitignore` 模板速查
- 11. GitHub Issue / Project / Wiki 协作
- 12. GitHub Actions 自动化入门
- 14. 其他 VCS 平台对比：GitHub / GitLab / Gitea / GitCode / Gitee / Bitbucket / SVN / Mercurial / Perforce
- 15. monorepo 是什么：单仓多包 / 多仓多包 / 工具选型
- 13. 小结与自检

---

1. Git 是什么？为什么必须学?

Git 是事实上的"代码时间机器"——每一次提交都是一个可回滚的版本快照；配合 GitHub / GitLab / Gitee 还能多人协作。

你将解决的事	没有 Git 怎么活
代码改坏了找不回来	复制 N 份"项目_最终版_真的最终.zip"
想看"上周三是哪个版本"	全靠记忆 / 没有
多人同时改一个文件	QQ 传文件互相覆盖 / 哭
自动跑测试 / 部署	手动 SSH 上服务器操作
接受开源社区贡献	邮件附件 patch

> **现实**：哪怕你一个人写代码，**也要用 Git**。否则你迟早会丢文件、记不清某段代码为什么改过、或者某天硬盘坏了。

1.1 三个核心名词先混个脸熟

- **仓库（Repository / repo）**：一个项目的所有文件 + 历史的总和
- **提交（Commit）**：一次完整的"修改快照"，有唯一哈希（如 `a1b2c3d`）
- **分支（Branch）**：从主线上"分叉"出来的独立工作流，合并后又能合回主线

---

2. 安装与首次配置

2.1 安装

平台	命令 / 链接
macOS	`xcode-select --install` 或 `brew install git`
Windows	https://git-scm.com/download/win（默认选项一路 Next）
Linux (Debian/Ubuntu)	`sudo apt install git`
Linux (CentOS/RHEL)	`sudo yum install git`

安装完成后打开终端验证：

```
git --version
# git version 2.42.0 或更高
```

2.2 首次配置

```
git config --global user.name "你的名字"
git config --global user.email "your@email.com"

# 让 `git log` / `diff` 显示颜色
git config --global color.ui auto

# Windows 用户必做：换行符处理，避免 CRLF 引发冲突
git config --global core.autocrlf input
# macOS / Linux 用户
git config --global core.autocrlf false

# 默认分支名（GitHub 2022 年起默认 main）
git config --global init.defaultBranch main
```

> **小知识**：上述配置存在 `~/.gitconfig`（Linux/macOS）或 `C:\Users\你\.gitconfig`（Windows）。`--global` 表示"对当前用户生效"。

2.3 `--global` 什么时候用 / 什么时候不用

> 刚学 Git 时所有人都用 `git config --global ...`，但**它不是万能的**。这一节说清楚 Git 配置的"三档作用域"，以及多账号场景下 `--global` 会带来的麻烦。

2.3.1 Git 配置的三档作用域

作用域	配置文件	作用范围	何时用
`--system`	`/etc/gitconfig`（Win：`C:\Program Files\Git\etc\gitconfig`）	这台电脑的**所有用户**	几乎不用（要管理员权限）
`--global`	`~/.gitconfig`	当前用户的**所有仓库**	99% 的日常配置（用户名、邮箱、换行符）

作用域	配置文件	作用范围	何时用
不带参数	`<仓库>/.git/config`	**当前这一个仓库**	覆盖 `--global` 的某些项

> **规律**：配置查找是"由近及远"——仓库级 → 全用户级 → 系统级，**近的覆盖远的。**

```
# 查看所有配置 + 来源
git config --list --show-origin

# 只看当前仓库生效的配置
git config --list --local
```

2.3.2 三档"使用 vs 不使用 --global"决策表

配置项	该用哪一档?	为什么
`user.name`	`--global`	一台电脑就一个名字
`user.email`	`--global`（**但多账号时例外**）	同上，**除非你有多个 Git 平台账号**
`init.defaultBranch`	`--global`	主分支名对所有仓库一致
`core.autocrlf`	`--global`	换行符规则对所有仓库一致
`color.ui`	`--global`	UI 偏好跟人走
`core.editor`	`--global`	个人编辑器偏好
`alias.*`	`--global`	命令别名跨仓库通用
`remote.origin.url`	**不带参数（仓库级）**	每个仓库的远程地址都不同
`branch.main.merge`	**不带参数（仓库级）**	每个仓库的上游分支不同
`user.email`（多账号时）	**不带参数 / 条件 include**	不同项目用不同邮箱
`commit.gpgsign`	看场景	公司项目可能要签名、个人项目不签

2.3.3 什么时候不用 `--global`

场景	原因	替代方案
公司项目要署名公司邮箱	不想把公司邮箱写进 `~/.gitconfig`	仓库级 `git config user.email "work@company.com"`
个人项目用 GitHub 邮箱、开源项目用 `noreply`	不想混	仓库级覆盖
多账号（一台电脑多个 GitHub 账号）	全局只有一个邮箱，多账号无法区分	**条件 include**（见 2.3.4）
只想临时改一项	不想动全局	仓库级配置
CI / Docker 容器内	容器内不需要持久化	命令行 `-c user.email=...` 临时传

2.3.4 多账号场景：用条件 include 代替 `--global`

> **核心思想**：根据**当前目录路径**，自动用不同的 Git 配置——这是多账号的唯一"正确解"。

`~/.gitconfig`（顶层）：

```
[user]
    name = 个人默认
    email = personal@gmail.com

# 公司项目放 ~/work/ 目录
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work

# 开源贡献放 ~/opensource/ 目录
[includeIf "gitdir:~/opensource/"]
    path = ~/.gitconfig-opensource
```

`~/.gitconfig-work`：

```
[user]
    name = 张三
    email = zhangsan@company.com
```

`~/.gitconfig-opensource`：

```
[user]
    name = tom-personal
    email = tom@users.noreply.github.com
```

效果：

```
cd ~/work/my-company-app
git config user.email    # → zhangsan@company.com

cd ~/opensource/my-lib
git config user.email    # → tom@users.noreply.github.com
```

> **判断顺序**：Git 从当前仓库路径**向上**匹配 `gitdir:`，**第一个命中**的 include 生效。

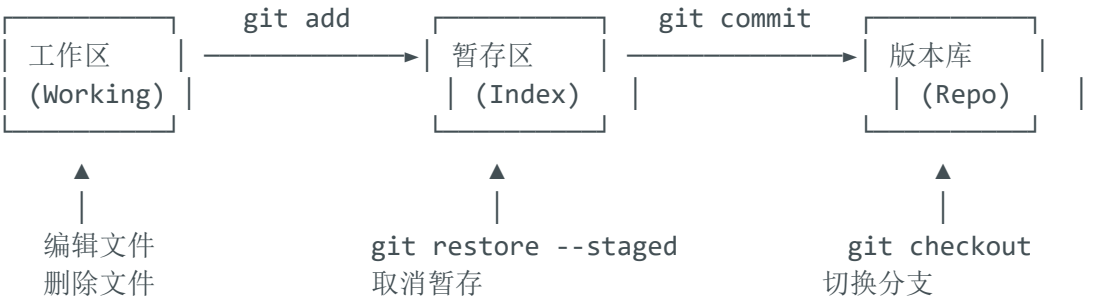
2.3.5 一句话

> **个人单账号**：放心用 `--global`；**多账号 / 多公司邮箱**：用 `includeIf "gitdir:..."` **按目录**自动切，**别在仓库里瞎用 `--global`**。

---

3. 核心概念：三个区、四种状态

把 Git 想成"有三条传送带"：



文件在这三个区之间有四种状态：

状态	含义	命令
未跟踪（Untracked）	新建的文件，Git 还没"管"它	`git add`
已修改（Modified）	改过但还没 `add`	`git add`
已暂存（Staged）	已 `add` 等着提交	`git commit`

状态	含义	命令
已提交（Committed）	已经写入历史	—

```
git status          # 看文件当前状态（最常用！）
```

---

4. 基础流程：本地 + 远程

4.1 把本地代码推到 GitHub

方式 A：本地先有项目

```
cd my-project
git init
git add .
git commit -m "feat: 初始化项目"
git branch -M main                  # 强制主分支叫 main
git remote add origin git@github.com:你的用户名/你的仓库.git
git push -u origin main             # -u 记住 upstream，以后直接 git push
```

方式 B：先在 GitHub 建仓库，再 clone 到本地

```
git clone git@github.com:你的用户名/你的仓库.git
cd 你的仓库
# 写代码...
git add .
git commit -m "feat: 第一次提交"
git push
```

4.2 SSH Key 配置（推荐，避免每次输密码）

```
# 1. 生成密钥（一路回车）
ssh-keygen -t ed25519 -C "your@email.com"

# 2. 复制公钥
cat ~/.ssh/id_ed25519.pub           # macOS / Linux
# Windows: type %userprofile%\.ssh\id_ed25519.pub

# 3. 粘贴到 GitHub：Settings → SSH and GPG keys → New SSH key
```

```
# 4. 验证
ssh -T git@github.com
# 看到 "Hi xxx! You've successfully authenticated..." 就 OK
```

> **HTTPS vs SSH**：新项目推荐 SSH；HTTPS 在某些代理下更稳，按需选。

4.3 多账号管理：一台电脑多个 Git 平台 / 多个同平台账号

> 你很可能会遇到：
>
> - 家里电脑既要推 GitHub 个人号、又要推 Gitee 公司号；
> - 工作电脑同时有"个人 GitHub + 公司 GitHub（GitHub Enterprise）+ 公司 GitLab"三套账号；
> - 帮客户改代码，用自己的 GitHub 推 PR，但客户 GitLab 也要能推。
>
> 一台电脑 + 一个 GitHub 默认 key **不够用**——这一节讲怎么用 `~/.ssh/config` + 多 key 解决，**Windows / Linux 各给完整例子**。

4.3.1 3 种典型场景

场景	怎么配
A. 一台电脑一个 GitHub 账号	4.2 那节的内容就够了
B. 一台电脑多个 GitHub 账号（个人 + 公司 Enterprise）	多 key + `~/.ssh/config` + `Host` 别名
C. 一台电脑多个平台账号（GitHub + GitLab + Gitee）	多 key + 每个平台一个 `Host` 别名
D. 同平台但想"按目录自动切"	`includeIf "gitdir:..."` 自动切 `user.email`（见 2.3.4）

> 现实里 **B + C + D 经常同时存在**：一台电脑、3 个平台、4 个账号、6 把 key——下面的例子就是这种"复杂但真实"的场景。

4.3.2 原理：SSH 怎么"认人"

```
ssh -T git@github.com
   |
   ▼
```

```
客户端拿 ~/.ssh/config 找匹配 Host
   |
   ▼
用对应 Host 的 IdentityFile（密钥）发起认证
   |
   ▼
GitHub 收到公钥 → 查是哪个账号加的 → 允许 / 拒绝
```

关键点：

- 一个 `~/.ssh/id_ed25519` 对应**一对**密钥（公钥 + 私钥）；
- 一个账号可以加**多把**公钥（公司电脑 / 家里电脑各一把）；
- 一台电脑可以有**多把**私钥，靠 `~/.ssh/config` 的 `Host` 别名切换。

4.3.3 准备：生成 4 把 key（个人 / 公司 GitHub / GitLab / Gitee）

> 一个平台一把 key，**命名要见名知意**。

Linux / macOS（bash / zsh）：

```
# 1. 个人 GitHub
ssh-keygen -t ed25519 -C "your@personal.com" \
  -f ~/.ssh/id_ed25519_github_personal -N ""

# 2. 公司 GitHub Enterprise
ssh-keygen -t ed25519 -C "your@company.com" \
  -f ~/.ssh/id_ed25519_github_work -N ""

# 3. 公司 GitLab
ssh-keygen -t ed25519 -C "your@company.com" \
  -f ~/.ssh/id_ed25519_gitlab -N ""

# 4. Gitee
ssh-keygen -t ed25519 -C "your@gitee.com" \
  -f ~/.ssh/id_ed25519_gitee -N ""

# 验证 4 把都生成了
ls -la ~/.ssh/id_ed25519_*
```

Windows（PowerShell）：

```
# 同上 4 条命令，但路径用 $HOME\.ssh\
ssh-keygen -t ed25519 -C "your@personal.com" `
  -f "$HOME\.ssh\id_ed25519_github_personal" -N ""

ssh-keygen -t ed25519 -C "your@company.com" `
  -f "$HOME\.ssh\id_ed25519_github_work" -N ""

ssh-keygen -t ed25519 -C "your@company.com" `
```

```
  -f "$HOME\.ssh\id_ed25519_gitlab" -N ""

ssh-keygen -t ed25519 -C "your@gitee.com" `
  -f "$HOME\.ssh\id_ed25519_gitee" -N ""

dir $HOME\.ssh\id_ed25519_*
```

> Windows 用 `dir` 看，`-N ""` 表示空密码（CI 友好）。**私钥文件不要给别人看。**

4.3.4 把 4 个公钥分别贴到 4 个平台

平台	路径
GitHub 个人	Settings → SSH and GPG keys → New SSH key，Title 写"personal mac"
GitHub Enterprise	同上，但登录的是 Enterprise 账号
GitLab	Preferences → SSH Keys
Gitee	设置 → SSH 公钥

```
# Linux / macOS：分别打印公钥
cat ~/.ssh/id_ed25519_github_personal.pub
cat ~/.ssh/id_ed25519_github_work.pub
cat ~/.ssh/id_ed25519_gitlab.pub
cat ~/.ssh/id_ed25519_gitee.pub
```

```
# Windows：分别打印公钥
type $HOME\.ssh\id_ed25519_github_personal.pub
type $HOME\.ssh\id_ed25519_github_work.pub
type $HOME\.ssh\id_ed25519_gitlab.pub
type $HOME\.ssh\id_ed25519_gitee.pub
```

每条输出都是一串 `ssh-ed25519 AAAA...`，**整段复制**贴到对应平台。

4.3.5 写 `~/.ssh/config`：核心步骤

> SSH `config` 文件是"按 Host 别名选不同 key"的唯一入口。**没有它，4 把 key 没法自动切换。**

Linux / macOS：

```
cat > ~/.ssh/config <<'EOF'
# ================== 个人 GitHub ==================
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github_personal
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================== 公司 GitHub Enterprise ==================
# 假设 Enterprise 域名是 github.acme.com
Host gh.acme
    HostName github.acme.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github_work
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================== 公司 GitLab ==================
# 假设 GitLab 域名是 gitlab.acme.com
Host gl.acme
    HostName gitlab.acme.com
    User git
    IdentityFile ~/.ssh/id_ed25519_gitlab
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================== Gitee ==================
Host gitee.com
    HostName gitee.com
    User git
    IdentityFile ~/.ssh/id_ed25519_gitee
    IdentitiesOnly yes
    AddKeysToAgent yes
EOF

# 权限必须是 600
chmod 600 ~/.ssh/config
```

Windows：

```
# PowerShell
$configPath = "$HOME\.ssh\config"
@"
# ================== 个人 GitHub ==================
Host github.com
    HostName github.com
    User git
    IdentityFile $HOME\.ssh\id_ed25519_github_personal
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================== 公司 GitHub Enterprise ==================
Host gh.acme
```

```
    HostName github.acme.com
    User git
    IdentityFile $HOME\.ssh\id_ed25519_github_work
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================= 公司 GitLab ==================
Host gl.acme
    HostName gitlab.acme.com
    User git
    IdentityFile $HOME\.ssh\id_ed25519_gitlab
    IdentitiesOnly yes
    AddKeysToAgent yes

# ================= Gitee ==================
Host gitee.com
    HostName gitee.com
    User git
    IdentityFile $HOME\.ssh\id_ed25519_gitee
    IdentitiesOnly yes
    AddKeysToAgent yes
"@ | Out-File -FilePath $configPath -Encoding utf8 -NoNewline
```

> 关键参数：
>
> - `Host`：别名（不是域名，是"你想叫它什么"）
> - `HostName`：真正的域名
> - `IdentityFile`：用哪把私钥
> - `IdentitiesOnly yes`：**只**用 `IdentityFile` 指定的 key，**不**自动尝试 `~/.ssh/id_rsa` 等默认 key（避免和别的账号"撞车"）
> - `AddKeysToAgent yes`（macOS 必需）：把 key 加到 ssh-agent，免去每次输密码

4.3.6 验证 4 把 key 都能连上

Linux / macOS：

```
ssh -T git@github.com        # → Hi <个人用户名>! ...
ssh -T git@gh.acme            # → Hi <公司用户名>! ...
ssh -T git@gl.acme            # → Welcome to GitLab, @<公司用户名>!
ssh -T git@gitee.com          # → Hi <Gitee 用户名>! ...
```

Windows（PowerShell）：

```
ssh -T git@github.com
ssh -T git@gh.acme
```

```
ssh -T git@gl.acme
ssh -T git@gitee.com
```

> 第一次连会问 `Are you sure you want to continue connecting (yes/no)?` ——**输入 yes** 即可（确认公钥指纹）。

4.3.7 clone / push 时怎么用别名

```
# 拉公司项目
git clone git@gl.acme:my-team/my-project.git
#   └── 注意：HostName 是 gitlab.acme.com，但写时用别名 gl.acme

# 拉个人项目（直接用官方域名）
git clone git@github.com:your-name/your-repo.git

# 拉 GitHub Enterprise
git clone git@gh.acme:company-org/company-repo.git

# 拉 Gitee
git clone git@gitee.com:your-name/your-repo.git
```

关键：clone 命令里写 **`Host` 别名，不是** `HostName`。

4.3.8 同一平台多个账号：必须用别名 Host

> **最容易踩的坑**：默认 `github.com` **只能绑一把 key**。如果你有两个 GitHub 账号都要走 `git@github.com:xxx/yyy.git`，**第二个一定失败**。

解法：给第二个账号起**不同的 Host 别名**。

```
# 个人 GitHub：走官方域名
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github_personal
    IdentitiesOnly yes

# 公司 GitHub（用别名 ghe 占住）
Host ghe
    HostName github.com            # ← 注意：和上面 HostName 相同，但 Host 别名不同
    User git
    IdentityFile ~/.ssh/id_ed25519_github_work
    IdentitiesOnly yes
```

使用：

```
# 个人项目（直接用 github.com）
git clone git@github.com:tom/personal-repo.git

# 公司项目（用别名 ghe）
git clone git@ghe:acme-org/company-repo.git
```

> 看起来 `HostName` 都是 `github.com`，但**别名不同**——SSH 会按别名匹配 IdentityFile，从而用不同的 key。**这是同平台多账号的唯一正确解**。

4.3.9 Windows 用 Git Credential Manager（GCM）记 HTTPS 密码

> 如果你用 HTTPS 而不是 SSH，**别再把 token 写在 `~/.gitconfig` 里**——用 GCM 让系统帮你记。

安装：

- 已装 **Git for Windows** → 已有 GCM
- macOS：`brew install --cask git-credential-manager`
- Linux：见 https://github.com/git-ecosystems/git-credential-manager

第一次 push 时会弹窗：

```
Sign in with your browser to continue.
```

浏览器登录后 token 自动存到系统 Keychain / Credential Manager，**以后免输入**。

```
# 检查 GCM 是否生效
git config --global credential.helper
# Windows / macOS 应该输出 manager / manager-core
```

4.3.10 常见坑速查

现象	原因	解决
`Permission denied (publickey)`	key 路径写错 / 没用别名	检查 `~/.ssh/config` 的 `Host` 和 `IdentityFile`
`git@github.com: Permission denied` 一直提示	默认 key 还在被尝试	确认 `IdentitiesOnly yes`

现象	原因	解决
同平台两个账号都连 `github.com` 失败	一域名只能绑一把 key	给第二个账号起**别名**（见 4.3.8）
克隆公司仓库但 `git config user.email` 仍是个人邮箱	没用 `includeIf`	见 2.3.4
Windows 上 `~/.ssh/config` 不生效	路径不对 / 权限太松	路径必须 `%USERPROFILE%\.ssh\config`，文件不能有 `.txt` 后缀
改完 `~/.ssh/config` 没生效	SSH 缓存	跑 `ssh-add -D` 清空再 `ssh-add <key>` 重启 agent
同一 key 在多个 GitHub 账号上重复使用	平台不允许	每账号独立 key

4.3.11 验证清单（跑完下面 4 行才算"多账号配置完成"）

```
# 1. 四把 key 都连得上
ssh -T git@github.com   && \
ssh -T git@gh.acme       && \
ssh -T git@gl.acme       && \
ssh -T git@gitee.com

# 2. clone 一个个人项目能 push
git clone git@github.com:tom/repo.git && cd repo && \
echo "hi" >> README.md && git add . && git commit -m "test" && git push

# 3. clone 一个公司项目能 push
git clone git@gl.acme:my-team/repo.git && cd repo && \
echo "hi" >> README.md && git add . && git commit -m "test" && git push

# 4. 检查 commit 显示的是"对应账号"的邮箱
git log -1 --format='%an <%ae>'
# 在 ~/work/ 下应该是 zhangsan@company.com
# 在 ~/personal/ 下应该是 tom@gmail.com
```

4.3.12 一句话总结

> **多账号的核心是"按目录分 key + 按别名选 key"**——`~/.ssh/config` 把别名和私钥配对，`includeIf "gitdir:..."` 把目录和邮箱配对。两边配合，**任何账号都自动选对 key、选对邮箱**，不用每次手切。

---

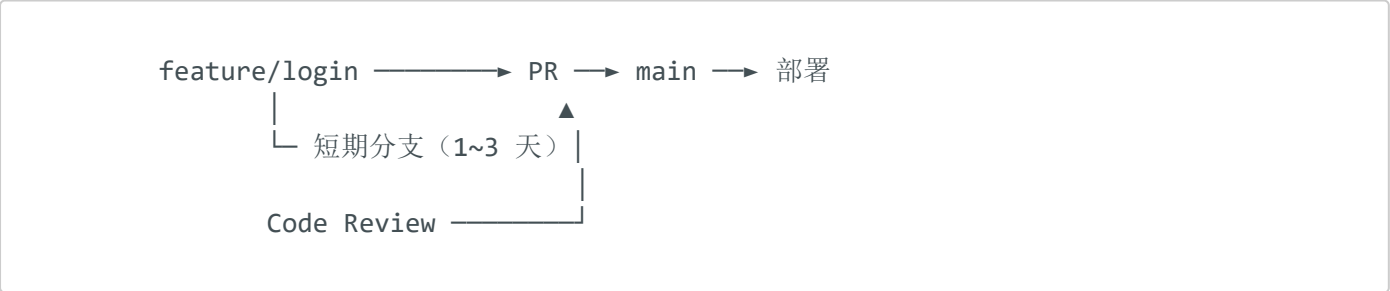
5. 分支策略：Trunk-Based / Git Flow / GitHub Flow

> "分支怎么管"是团队协作的核心问题。选错策略 = 每天在合并冲突里过日子。

5.1 三大主流分支模型

模型	主分支	辅助分支	适合谁	节奏
Trunk-Based	`main`	短期 feature 分支（< 1 天）	高频发版、CI/CD 成熟团队	每天合并到 main
GitHub Flow	`main`	任意命名的 feature 分支	中小团队 / SaaS / 互联网产品	PR → review → merge
Git Flow	`main` + `develop`	`feature/*` `release/*` `hotfix/*`	有"版本号"概念的桌面 / 嵌入式	每周/每月一版

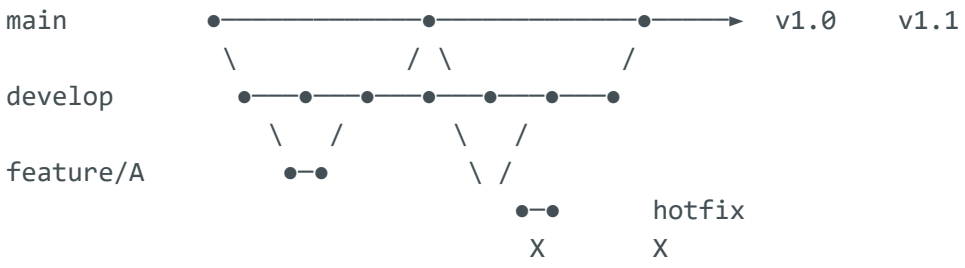
5.2 推荐：GitHub Flow（99% 的现代项目都用它）



核心规则：

1. `main` 分支**永远可发布**（CI 必须绿）
2. 任何新功能 / 修复 = 从 `main` 拉新分支
3. 分支上写完提 PR → 团队评审 → 合并回 `main`
4. 合并后**立即部署**

5.3 Git Flow（适合"发版有节奏"的传统项目）



- `main`：每个 tag 是一个发布版本
- `develop`：日常集成分支
- `feature/*`：从 develop 拉，合并回 develop
- `release/*`：发版前准备，测完合并回 main + develop
- `hotfix/*`：从 main 拉紧急修复

> **初学者不要硬套 Git Flow**——它流程重、合并分支多，对小项目是负担。

5.4 命名约定

```
# 推荐格式：<类型>/<简短描述>
feature/login              # 新功能
feature/user-profile
fix/header-404             # 修 bug
refactor/axios-instance    # 重构
docs/update-readme         # 文档
chore/bump-vite-6          # 杂项
```

> 类型对应 Conventional Commits 的 `feat / fix / refactor / docs / chore` 等。

---

6. 团队协作：Pull Request 与 Code Review

> **PR（Pull Request）** 是 GitHub 协作的核心：不是"代码合并请求"，而是"团队评审 + 讨论 + 留痕"的载体。

6.1 一次完整的 PR 流程

```
1. 从 main 拉分支          git checkout -b feature/login
2. 写代码 + 多次 commit
3. 推到远程                 git push origin feature/login
4. 在 GitHub 上开 PR        → 选 base = main, compare = feature/login
5. 写 PR 描述：改了什么 / 为什么改 / 截图 / 关联 Issue
6. 通知 reviewer            → 在 PR 右侧选 Reviewers
7. reviewer 看代码、写评论、approve / request changes
8. CI 自动跑测试            → 必须绿才能合并
9. 通过后 Squash and merge（推荐）
10. 删除远端 feature 分支
```

6.2 PR 描述模板 `.github/pull_request_template.md`

把下面这文件放到仓库根目录 `.github/` 下，下次开 PR 就会自动加载：

```
## 改了什么
<!-- 一句话讲清楚这一 PR 做了什么 -->

## 改动的类型
- [ ] 新功能（feature）
- [ ] Bug 修复（fix）
- [ ] 重构（refactor）
- [ ] 文档（docs）
- [ ] 杂项（chore）

## 怎么测
<!-- 步骤 + 截图 / 录屏 -->

## 关联 Issue
<!-- 关闭 / 关联 #xxx -->

## 自检清单
- [ ] 跑过 `npm run dev` 验证
- [ ] 跑过 `npm run build` 没有报错
- [ ] 新增 / 修改的代码有注释
- [ ] 没把 `console.log` 留在生产代码里
```

6.3 Code Review 的三个原则

原则	含义
就事论事	评论代码不评论人；用"这里可以……"代替"你怎么又……"
小步快跑	PR 越短越好；200 行以内的 PR 比 2000 行的 PR review 体验好 10 倍

原则	含义
及时响应	作者提了 PR 24 小时内至少回应；reviewer 收到通知尽快看

6.4 高频 review 关注点

- 命名是否表意（`btn1` / `data2` ❌；`submitButton` / `userList` ✅）
- 是否引入了不必要的依赖
- 错误处理是否到位（有没有 `catch` 漏掉的）
- 是否有 console.log / debugger 残留
- 类型是否完整（TS 项目）
- 测试是否覆盖关键路径

6.5 合并方式：merge / squash / rebase

方式	行为	适合
Merge commit	保留所有 commit + 一个 merge commit	想保留完整历史
Squash and merge	把所有 commit 压成一个	**最常用**，main 历史干净
Rebase and merge	把分支 commit 接到 main 之后	想保留 commit 但不要 merge commit

> **推荐默认 Squash and merge**，让 `main` 上每个 commit 都是一个独立可回滚的"功能单元"。

---

7. 冲突处理：什么时候 rebase，什么时候 merge

7.1 rebase vs merge 速记

```
# 把当前分支"接到" main 最新提交之后
git rebase main
```

```
# 把 main 合并到当前分支（保留所有历史）
git merge main
```

维度	rebase	merge
历史	干净的直线	有 merge commit 节点
冲突处理时机	一次性、容易	每个冲突 commit 都要处理
适合	**本地**整理未推送的 commit	推送后、合并重要 feature

7.2 黄金规则

> **没推上去的 commit：随便 rebase；已经推上去的 commit：永远别 rebase。**

否则你的同事 `git pull` 后会发现历史对不上号，全员崩溃。

7.3 实战：解决合并冲突

```
git pull origin main         # 或 git fetch + git rebase origin/main
# CONFLICT (content): Merge conflict in src/utils/request.ts
git status                   # 看哪些文件冲突
# 打开冲突文件，找到 <<<<<<< HEAD ======= >>>>>>> main 标记，手动选择保留谁
git add src/utils/request.ts
git commit                    # 合并冲突用 git commit（不是 git rebase --continue 那
种）
```

7.4 后悔药

```
git rebase --abort           # rebase 过程中想撤销
git merge --abort            # merge 过程中想撤销
git reset --hard HEAD~1      # 撤销最近一次 commit（**未推送才用**）
```

---

8. 提交规范：Conventional Commits

让 commit 信息**机器可读**，自动生成 CHANGELOG。

8.1 格式

```
<类型>(<范围>): <简短描述>

<body>（可选，72 字符换行）

<footer>（可选，关联 Issue）
```

8.2 常用类型

类型	含义	例子
feat	新功能	feat(login): 新增手机号登录
fix	修 bug	fix(header): 修复移动端导航错位
docs	文档	docs(readme): 补充部署步骤
style	格式（不改逻辑）	style: 统一代码缩进
refactor	重构	refactor(api): 抽出 request 工具方法
perf	性能	perf(list): 虚拟滚动优化 10000 项
test	测试	test(button): 补点击事件单测
chore	杂项	chore: 升级 vite 到 6.0
ci	CI 配置	ci: 添加 GitHub Actions
revert	回滚	revert: 撤回 feat(login)

8.3 实战

```
git commit -m "feat(login): 新增手机号登录"
git commit -m "fix(header): 修复移动端导航错位

之前的 z-index 在 iOS Safari 失效
原因：没有 -webkit-overflow-scrolling: touch

Closes #123"
```

8.4 工具推荐

- **commitlint + husky**：提交时自动校验格式
- **standard-version / release-please**：自动生成 CHANGELOG 和版本号

---

9. 高频救命命令：找回误删 / 回退版本 / 改 commit 历史

> 这一节是"代码丢了怎么办"的速查表，建议收藏。

9.1 误删文件 / 误删分支

```
# 误删文件：恢复成上次 commit 的样子
git checkout -- path/to/file

# 误删分支：reflog 里找
git reflog                              # 列出所有 HEAD 移动记录
# 找到那个分支最后一次的 commit hash
git checkout -b feature/login <hash>    # 重新拉回来
```

> **黄金法则**：`git reflog` 能救回 30 天内几乎所有"误操作"——只要你**已经 commit 过**。

9.2 撤销最近一次 commit

场景	命令
撤销 commit，**保留**改动在工作区	`git reset --soft HEAD~1`
撤销 commit，**清空**改动	`git reset --hard HEAD~1` ⚠️
改写最近一次 commit 信息	`git commit --amend`
已经推送？补一个"反向" commit	`git revert HEAD` 然后 push

> `--hard` 不可恢复！未推送时谨慎用，已推送时**绝对不要用**。

9.3 改写历史：rebase -i

```
git rebase -i HEAD~5   # 编辑最近 5 个 commit
```

弹出编辑器，把 `pick` 改成：

动作	含义
`pick`	保留
`reword`	改 commit 信息
`edit`	停下来改文件
`squash`	合并到上一个
`drop`	删掉

> 适合"准备发版前把 5 个 wip commit 整理成 1 个干净的"。

9.4 暂存当前工作

```
git stash                  # 暂存所有未提交改动
git stash pop              # 恢复
git stash list             # 查看所有暂存
git stash apply stash@{0}  # 恢复指定一个
```

> 适用场景：正在改一半被叫去修紧急 bug，先把当前改动"藏"起来。

9.5 GPG 签名 commit：公司项目 / 关键开源项目必备

> GitHub / GitLab 都支持"verified"绿色小标签——靠 **GPG（PGP 的 GNU 实现）签名**。公司项目、Linux 内核等关键项目**强制要求**。

9.5.1 一次配置永久使用

```
# 1. 生成 GPG key（一直回车 = 默认 + 不设密码）
gpg --full-generate-key
# 选 RSA and RSA，4096 bits，0 = key 永不过期

# 2. 列出你的 GPG key 找到 ID
gpg --list-secret-keys --keyid-format=long
```

```
# 输出类似：
#   sec   rsa4096/ABC123DEF456 2026-01-01
#                       ^^^^^^^^^^  ← 这就是 GPG key ID

# 3. 告诉 git 用这个 key 签名
git config --global user.signingkey ABC123DEF456
git config --global commit.gpgsign true              # 每次 commit 自动签
git config --global tag.gpgsign true                 # tag 也签

# 4. 导出公钥贴到 GitHub：Settings → SSH and GPG keys → New GPG key
gpg --armor --export ABC123DEF456
```

9.5.2 验证 commit 已签名

```
git log --show-signature -1
# commit abc123 (HEAD -> main)
# Author: Tom <tom@example.com>
# Date:   ...
#
# commit abc123...
# gpg: Signature made ...
# gpg: Good signature from "Tom <tom@example.com>" [ultimate]
```

9.5.3 常见问题

现象	解决
commit 后 GitHub 显示"Unverified"	没把 GPG 公钥贴到 GitHub，或 email 与 `user.email` 不一致
`gpg: signing failed: Inappropriate ioctl for device`	没启动 `gpg-agent`，先 `echo "test"`
想对**单次** commit 签名	`git commit -S -m "msg"`（**大写** -S）
已经 commit 没签，怎么补签	`git commit --amend -S --no-edit`
想跳过签名	`git commit --no-gpg-sign -m "wip"` 或临时 `git config commit.gpgsign false`

> **何时必须开：**
>
> - 任何商业 / 政企项目
> - 开源项目想被 GitHub 标 "verified"
> - 监管 / 审计要求"能证明代码是某某人提交"

> **何时可省**：个人学习项目 / 内部小工具 / 与现有 GPG 体系冲突时。

9.6 ssh-agent：免去每次输入 SSH 密码

> 4.2 教你用 SSH key 推代码——但**默认每次 push 都要输一次密钥密码**。`ssh-agent` 帮你"在系统里记住"，**整个会话只输一次**。

9.6.1 启动 agent + 把私钥加进去

```
# macOS（已自带）
ssh-add --apple-use-keychain ~/.ssh/id_ed25519_github_personal

# Linux 桌面
eval "$(ssh-agent -s)"              # 启动 agent
ssh-add ~/.ssh/id_ed25519_github_personal

# Windows（PowerShell）
Start-Service ssh-agent             # 设为自动启动
Set-Service -Name ssh-agent -StartupType Automatic
ssh-add $HOME\.ssh\id_ed25519_github_personal
```

之后 `git push` **永远不再问密码**，直到下次开机。

9.6.2 验证 agent 里的 key

```
ssh-add -l                          # 列出已加的 key 指纹
ssh-add -L                          # 列出完整公钥
```

9.6.3 macOS 把密码保存到 Keychain

```
# 第一次 ssh-add 时加 --apple-use-keychain
# 之后重启也不用重输—密码在 Keychain
```

> **Windows** 把密码存到 Credential Manager（第 4.3.9 节 Git Credential Manager同一机制）。

9.7 常见错误速查（10 大经典报错）

> 80% 的 Git 错误都在这 10 条里——**收藏**。

错误	原因	解决
`fatal: not a git repository`	在非 Git 目录里跑 git 命令	`cd` 到仓库，或 `git init`
`Permission denied (publickey)`	SSH key 没配 / 没加到 GitHub	见 4.2 SSH Key 配置
`fatal: refusing to merge unrelated histories`	两个无关仓库要合并	`git pull origin main --allow-unrelated-histories`（**慎用**）
`error: Your local changes would be overwritten`	拉代码会覆盖你的未提交改动	先 `git stash` 再 pull，最后 `git stash pop`
`Updates were rejected because the tip of your current branch is behind`	远端有 commit，你本地没有	先 `git pull`（或 `git pull --rebase`）再 push
`fatal: The current branch main has no upstream branch`	第一次 push 没指定远端	`git push -u origin main`
`error: failed to push some refs to ...`	远端有更新，push 被拒	先 pull 再 push（见上）
`CONFLICT (content): Merge conflict in xxx`	merge / rebase 时有冲突	打开文件找 `<<<<<<<` 标记手动改完，再 `git add` + commit
`fatal: refusing to merge unrelated histories`	同上	同上
`warning: LF will be replaced by CRLF`	Windows 上检出时自动换行	加 `.gitattributes`（见 10.4）
`error: src refspec xxx does not match any`	本地没有这个分支或 tag	`git branch -a` 看有什么分支
`fatal: unable to access ... Could not resolve host`	网络 / DNS / VPN 问题	切镜像（见 10.7）
`fatal: refusing to accept non-linear history`	push 时远端不允许非快进合	`git pull --rebase` 后再 push，或用 `git push --`

错误	原因	解决
	并	`force-with-lease`（**慎用**）

> **黄金法则：报错信息第一行几乎永远告诉你原因——先读，再搜。**

---

10. `.gitignore` 模板速查

新项目第一件事就是写好 `.gitignore`：

10.1 Node / 前端通用模板

```
# 依赖
node_modules/
.pnpm-store/

# 构建产物
dist/
build/
*.tsbuildinfo

# 环境变量
.env
.env.local
.env.*.local

# 日志
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*

# 编辑器
.vscode/
.idea/
*.swp
.DS_Store

# 测试
coverage/
.nyc_output/

# 缓存
.cache/
.parcel-cache/
.eslintcache
```

> **永远不要把 `node_modules` 提交到 Git**——它能被 `package-lock.json` / `pnpm-lock.yaml` 完整还原。

10.2 怎么生成

- gitignore.io：选技术栈自动生成
- VSCode 装 **gitignore** 插件：新建文件时模板自动出现

10.3 提交了不该提交的文件怎么办

```
# 从仓库里删掉，但保留本地文件
git rm --cached path/to/file
git commit -m "chore: 把 .env 从仓库移除"
```

> 重要：**已经在历史里的敏感信息**（如不小心提交了 `.env`），单靠 `git rm --cached` 不够——历史里还能看到。要用 git-filter-repo 真正擦除，**而且必须立刻更换密钥**。

10.4 `.gitattributes`：跨平台换行符 / LFS 全靠它

> 很多人只听过 `.gitignore`，但**漏配 `.gitattributes` 是仓库"莫名其妙改文件"的根因**。

常见用途 1：统一换行符（避免 Windows / macOS / Linux 互相污染）

```
# 文本文件：检出时转 LF，提交时转 LF
* text=auto eol=lf

# 强制 CRLF（Windows 旧项目）
# *.bat text eol=crlf

# 排除二进制（不要去碰它）
*.png binary
*.jpg binary
*.zip binary
```

常见用途 2：导出时排除某些文件

```
# 导出 zip / tar 时不包含测试和文档
/.github export-subst
/docs/      export-ignore
*.test.js  export-ignore
```

常见用途 3：搭配 Git LFS 存大文件（游戏 / 影视 / 机器学习）

```
*.psd filter=lfs diff=lfs merge=lfs -text
*.zip filter=lfs diff=lfs merge=lfs -text
*.onnx filter=lfs diff=lfs merge=lfs -text
```

> **口诀：跨平台团队第一天就该有 `.gitattributes`**——能省下 50% 的"为啥这个文件状态变了"。

10.5 Git Submodules：嵌套仓库

> 当你需要"项目里嵌一个独立仓库"（如把共享库、第三方代码用 Git 跟踪）时，**用 submodule**。

10.5.1 添加 submodule

```
# 把 https://github.com/xxx/shared-lib 作为本仓库的 submodules/shared
git submodule add https://github.com/xxx/shared-lib submodules/shared

# 之后 git status 会看到：
# new file:   .gitmodules
# new file:   submodules/shared
```

`.gitmodules` 记录了"哪里有什么子仓库"：

```
[submodule "submodules/shared"]
    path = submodules/shared
    url = https://github.com/xxx/shared-lib.git
```

10.5.2 clone 包含 submodule 的仓库

```
# 一行：把父仓库 + 所有 submodule 一起拉下来
git clone --recurse-submodules https://github.com/xxx/main-app.git
```

```
# 已经 clone 了？补救：
git submodule init
git submodule update
```

10.5.3 更新 submodule

```
# 1. 进 submodule 切到要用的分支 / 拉新代码
cd submodules/shared
git checkout main
git pull

# 2. 回到父仓库，commit 这个"子模块版本变更"
cd ../..
git add submodules/shared
git commit -m "chore: bump shared-lib to v1.2.3"
```

10.5.4 何时用 vs 何时不用

场景	推荐
公共库多项目共享	✅ monorepo（见 第 15 章）
第三方代码想跟上游同步	✅ submodule
需要精确控制子仓库版本	✅ submodule
频繁改子仓库代码	❌ 太麻烦
CI 构建慢	❌ submodule 的 clone 经常很慢

> **替代方案**：现在大多数场景用 **monorepo + pnpm workspace / Turborepo**（见 15 章），submodule 用得越来越少。

10.6 浅克隆与部分克隆：大仓库的救星

> 仓库动辄 1–10 GB（chrome、vue-next、tensorflow …）？**完整 clone 一次半小时——用浅克隆。**

10.6.1 `--depth` 浅克隆

```
# 只拉最近 1 次 commit
git clone --depth 1 https://github.com/vuejs/core.git
```

```
# 拉最近 50 次
git clone --depth 50 https://github.com/vuejs/core.git
```

> **代价**：拿不到完整历史，`git log` 只能看最近 N 条。**但 99% 的项目"只看最新代码"足够。**

10.6.2 `--filter=blob:none` 部分克隆（partial clone）

```
# 拿到所有 commit 和 tree，**但不下载任何文件**
git clone --filter=blob:none https://github.com/vuejs/core.git

# 第一次 checkout 时才真正下载文件（按需）
```

> **比 `--depth` 更强**：保留完整历史，但省下"大文件"的下载。

10.6.3 `--sparse` 稀疏检出：只拉一个子目录

```
# clone 后只检出 packages/react 一个目录
git clone --filter=blob:none --sparse https://github.com/facebook/react.git
cd react
git sparse-checkout set packages/react

# 想看全部？
git sparse-checkout disable
```

> **实战**：公司 monorepo 几 GB 时，**新员工只拉自己负责的包**，clone 10 秒搞定。

10.6.4 一句话选型

你想	用这个
拉代码就够，不看历史	`git clone --depth 1`
拉所有 commit 但不下载大文件	`git clone --filter=blob:none`
只看一个子目录	`--sparse` + `sparse-checkout set`
真的需要完整历史	**老老实实** `git clone`

10.7 国内镜像加速：clone GitHub 不再卡死

> 国内 clone GitHub 平均 10–50 KB/s，10 GB 仓库要等**大半天**。**用镜像**。

10.7.1 GitHub 官方镜像

```
# 把 github.com 换成 ghproxy.com（GitHub 官方反代镜像）
git clone https://ghproxy.com/https://github.com/vuejs/core.git

# 或用 gh-proxy.com
git clone https://gh-proxy.com/https://raw.githubusercontent.com/...
```

> **注意**：镜像只是"代理"，**权限控制**和**私密仓库**还是走你自己电脑的 SSH key，不会经镜像泄露。

10.7.2 把镜像设成 git 全局配置（推荐）

```
# 国内用户推荐：把 https 替换走镜像
git config --global url."https://ghproxy.com/https://github.com/".insteadOf
"https://github.com/"
```

之后 `git clone https://github.com/xxx/yyy.git` 自动走镜像。

10.7.3 第三方镜像站速查

镜像	域名	适合
ghproxy	`https://ghproxy.com`	通用反代（**最常用**）
mirror.ghproxy.com	`https://mirror.ghproxy.com`	ghproxy 镜像
kkgithub	`https://kkgithub.com`	通用
fastgit	`https://hub.fastgit.xyz`	通用
阿里云 npm 镜像	`https://registry.npmmirror.com`	**只镜像 npm**（不是 git）

> **重要**：**别把镜像写进 push URL**——push 走镜像会被 ban。镜像只用于 **clone**。

10.7.4 自部署 git 镜像

公司 / 学校内网常用 `git clone` 内网仓库速度慢，可在 CI / 内网 GitLab 上挂"自动同步 GitHub"：

```
# GitLab 自带的"镜像仓库"功能
```

```
# 或用 gitea 做 GitHub 反代（mirror）
```

> **一句话：国内 clone 必配镜像**——`ghproxy.com` 几乎能解决 80% 的痛点。

---

11. GitHub Issue / Project / Wiki 协作

> 写代码只是软件开发的一半；另一半是"沟通、跟踪、复盘"。GitHub 提供一整套工具让这些事不必靠微信群。

11.1 Issue：轻量任务管理

典型用法：

- 报 bug
- 提需求 / 改进建议
- 提问题求帮助
- 任务分派

Issue 模板 `.github/ISSUE_TEMPLATE/bug.md`：

```
---
name: Bug 报告
about: 创建一个报告来帮助我们改进
title: '[BUG] '
labels: bug
---

## 复现步骤
1. 打开 ...
2. 点击 ...
3. 看到 ...

## 期望行为
<!-- 期望是什么 -->

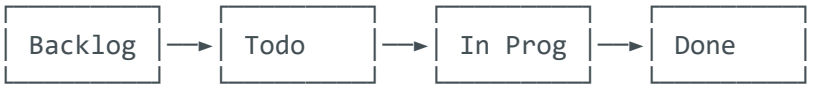
## 实际行为
<!-- 实际看到什么 -->

## 环境
- OS: Windows 11 / macOS 14 / Ubuntu 22.04
- 浏览器: Chrome 120
- 版本: v1.2.3
```

> **关键字关联**：在 PR 或 commit 信息里写 `Closes #123` / `Fixes #45`，合并 PR 后会自动关闭对应 Issue。

11.2 Project：看板视图

GitHub Project 是个**看板**——把 Issue / PR 拖到不同列：



适合 3-10 人小团队代替 Jira。

11.3 Discussion：问答 / 公告

比 Issue 更轻量——适合"我想问个问题"或"我们发了 v2.0 大家看 README"。

11.4 Wiki：长期文档

适合放**不会频繁变动**的内容：架构图、API 文档、Onboarding 指南。本教程 README 也是类似角色。

11.5 三者对比

工具	适合	寿命
Issue	任务、Bug、具体功能	完成后关闭
Project	团队看板上视图	长期存在
Discussion	问答、公告、闲聊	长期存在
Wiki	长期文档	长期存在

---

12. GitHub Actions 自动化入门

> **CI（持续集成）/ CD（持续部署）** 听着高大上，本质就是"每次 push / PR，机器自动跑你写好的脚本"。GitHub Actions 是 GitHub 内置的 CI 工具。

12.1 核心概念

名词	含义
Workflow	一个 YAML 文件，定义"什么事件触发 + 做什么事"
Job	一个工作单元（如"跑测试" / "打包"）
Step	Job 里的一步命令
Action	别人写好的可复用步骤（`actions/checkout@v4` 等）
Runner	真正执行命令的虚拟机（GitHub 免费提供 Linux 2 核 7G）

12.2 文件位置

`.github/workflows/<任意名>.yml`

12.3 最小可运行示例：跑测试

```yaml
# .github/workflows/test.yml
name: Test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: 装 Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'

      - name: 装 pnpm
        run: npm install -g pnpm
```

```yaml
      - name: 装依赖
        run: pnpm install --frozen-lockfile

      - name: 跑测试 + 覆盖率
        run: pnpm test -- --coverage

      - name: 上传覆盖率到 Codecov
        uses: codecov/codecov-action@v4
        with:
          token: ${{ secrets.CODECOV_TOKEN }}
```

12.4 部署到服务器：SSH + rsync

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]      # 推 main 就部署

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: 用 Node 20 构建
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'

      - run: pnpm install --frozen-lockfile
      - run: pnpm build

      - name: 把 dist/ 拷到服务器
        uses: appleboy/scp-action@v0.1.7
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          source: "dist/*"
          target: "/var/www/my-app"

      - name: 在服务器上重启
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            cd /var/www/my-app
            pm2 restart my-app
```

> `secrets` **怎么加**：仓库 → Settings → Secrets and variables → Actions → New repository secret。每个密钥独立使用、独立轮换。

12.5 5 个最常用的现成 Action

Action	用途
`actions/checkout@v4`	拉代码
`actions/setup-node@v4`	装 Node

Action	用途
`actions/setup-python@v5`	装 Python
`appleboy/ssh-action@v1`	远程执行命令
`docker/build-push-action@v5`	构建并推送 Docker 镜像

> 浏览更多：https://github.com/marketplace?type=actions

12.6 实战节奏

```
push / PR
   │
   ▼
[CI] install → lint → test → build
   │
   ▼ (全绿才能合并)
合并到 main
   │
   ▼
[CD] build → deploy
```

---

14. 其他 VCS 平台对比：GitHub / GitLab / Gitea / GitCode / Gitee / Bitbucket / SVN / Mercurial / Perforce

> 前面所有章节都在讲 **Git + GitHub**——但 "Git" 是协议 / 工具，"GitHub" 只是其中一个托管平台。**它们不是一回事**：你可以用 Git 把代码托管到任何地方。本节一次性把常见的 9 个平台 / 系统摆出来对比，让你知道"为什么公司不让我用 GitHub"、"Gitee 和 GitCode 区别在哪"、"SVN 真的要被淘汰了吗"。

14.1 先分清两个概念

概念	含义	关系
VCS（Version Control System）	版本控制系统本身	**协议 / 工具**（Git、SVN、Mercurial、Perforce …）

概念	含义	关系
Forge / Code Hosting	托管代码 + 提供 PR / Issue / CI 等协作能力的**网站**	**平台**（GitHub、GitLab、Gitee、Gitea …）

> **Git 是工具，GitHub 是网站**。你可以用 Git 把代码托管在 GitHub、GitLab、Gitee、企业内网 Gitea，或者就放在自己的笔记本硬盘上——VCS 和"放在哪"是**正交**的两件事。

14.2 九大平台 / 系统速览

平台 / 系统	类型	出身 / 持有方	部署方式	主要特色	谁在用
GitHub	托管平台	美国 Microsoft，2008	SaaS（公有云）	全球最大开源社区、Copilot、Codespaces、Actions	全球开发者 / 开源项目
GitLab	平台 + 自部署	美国 GitLab Inc.（俄裔创始人），2011	SaaS + 自部署（CE/EE）	**内置 CI/CD 一体化**、安全合规最强、可离线	企业 / 政府 / 金融 / 想"一套全包"的团队
Gitea	自部署平台	开源社区（中国开发者发起），2016	**自托管**（轻量 Go 写）	极轻量、单二进制、UI 像 GitHub	中小企业 / 学校 / 个人自建
Gogs	自部署平台	开源（与 Gitea 同作者早期作品）	自托管（更轻量）	比 Gitea 还轻、但功能较少	极小团队 / 个人
GitCode	托管平台	**CSDN 旗下**，2023 推出	SaaS（公有云，国内）	整合 CSDN 流量、面向中文开发者、内置 AI 助手	国内个人 / 高校 / 国产开源

平台 / 系统	类型	出身 / 持有方	部署方式	主要特色	谁在用
Gitee（码云）	托管平台	**OSChina 旗下**，2013	SaaS + 企业版（私有化）	国内访问快、对中文名 / 邮箱友好、政企客户多	国内中小团队 / 政企 / 学生
Bitbucket	托管平台	澳大利亚 Atlassian，2008	SaaS + 自部署（DC）	**和 Jira / Confluence 原生打通**、Atlassian 生态	公司全用 Atlassian 工具栈
SVN（Subversion）	集中式 VCS	Apache 基金会，2000	工具（需自配服务端）	**集中式**版本控制、有"全局版本号"、权限细到目录	老牌国企 / 银行 / 游戏客户端 / Unity 老项目
Mercurial（hg）	分布式 VCS	Matt Mackall，2005	工具（命令行）	命令和 Git 极像、性能稳定，**已被 Bitbucket 弃用**	几乎只剩历史项目
Perforce / Helix Core	集中式 VCS	Perforce Software	商业（自部署为主）	**巨型二进制文件**、美术资产、游戏源码同时管	3A 游戏、影视特效、汽车 ECU、芯片

> **2026 年现状速记：**
>
> - 开源 / 个人 → **GitHub** + 偶尔 **Gitee / GitCode** 同步
> - 国内企业 / 金融 / 政府 → **GitLab 自部署** 或 **Gitee 企业版**
> - 想轻量自建 → **Gitea**（Gogs 已被 Gitea 超越）
> - 老项目 / 行业硬性要求 → **SVN** 或 **Perforce**
> - Mercurial：基本只剩历史包袱，新项目**别选**

14.3 三大平台功能对比

维度	GitHub	GitLab	Gitea	Gitee	GitCode
免费私有仓库	✅	✅	✅（自部署）	✅	✅
PR / MR	Pull Request	Merge Request	Pull Request	Pull Request	Pull Request
内置 CI/CD	GitHub Actions	**GitLab CI（一体化）**	Gitea Actions	Gitea Actions（兼容）	内置
Issue / Wiki	✅	✅	✅	✅	✅
Package Registry	npm / Maven / Docker	npm / Maven / Docker / Generic	基础	基础	基础
AI 辅助	Copilot	Duo	无	无	CodeGeeX / 内置 AI
国内访问速度	❌ 慢/被墙	⚠️ 自部署可控	✅ 自部署可控	✅ 快	✅ 快
价格（企业）	中	中-高	**免费**（自部署）	中	中
适合规模	个人 / 中小 / 开源	**大企业**	**中小团队自建**	国内中小 / 政企	国内个人 / 高校

14.4 Git vs SVN：两种 VCS 的根本分歧

> 这是面试常考题。**Git 是分布式（DVCS），SVN 是集中式（CVCS）**——这个根本差异决定一切。

维度	Git	SVN
架构	**分布式**：每人本地一份完整历史	**集中式**：所有人共用一个中央服务器
是否能离线提交	✅ 强（本地有完整历史）	❌ 必须连服务器
提交速度	秒级	取决于网络

维度	Git	SVN
分支	**极轻量**（指针 + 几个对象）	笨重（拷一份目录）
全局版本号	❌ 哈希号	✅ 有 `r1234` 这样的递增版本号
权限控制	较粗（仓库 / 分支级）	**细到目录 / 文件级**
大文件 / 二进制	友好（Git LFS）	原生支持
学习曲线	中（命令多）	低（命令少）
当前生态	**主流**（90%+ 新项目）	维护老项目 / 强权限场景

4 个核心区别用一句话说清

1. **离线**：Git 能本地提交，SVN 不能；
2. **分支**：Git 分支是"瞬移"，SVN 分支是"复制目录"；
3. **版本号**：Git 是 `a1b2c3d` 哈希，SVN 是 `r1234` 递增；
4. **权限**：SVN 能精确到"某人能不能改这个文件"，Git 一般只能"能不能 push 这个仓库"。

为什么 SVN 还没死

- **金融 / 政府** 对审计要求高：SVN 的"全局版本号"在合同 / 法规里是证据，Git 的哈希审计起来麻烦；
- **游戏客户端**：Unity 老项目、美术资产动辄几个 G，SVN 集中式 + 大文件原生支持更稳；
- **嵌入式 / 芯片**：某些工厂流程系统只支持 SVN；
- **老员工**：大量"会 SVN 不会 Git"的技术骨干。

> **简单记**：新项目几乎 100% 选 Git；遇到"公司明文规定用 SVN"再学不迟——基础概念 2 小时能上手。

14.5 Git vs Mercurial vs Perforce：三大特殊玩家

维度	Git	Mercurial（hg）	Perforce / Helix Core
架构	分布式	分布式	**集中式**（客户端轻、服务端重）
性能	仓库大时变慢	比 Git 略快	**巨型仓库（百万文件）依然秒级**

维度	Git	Mercurial（hg）	Perforce / Helix Core
大文件	Git LFS（外挂）	不擅长	**原生 + 强（游戏 / 影视行业标准）**
当前地位	主流	已式微	**3A 游戏 / 影视 / 汽车行业事实标准**
学习曲线	中	易	难（服务端运维复杂）

> **什么时候考虑非 Git 工具?**
>
> - 单仓库 > 10 万个文件 / 总大小 > 50 GB → 考虑 **Perforce**;
> - 公司历史包袱或甲方要求 → 选 **SVN**;
> - 几乎找不到理由选 **Mercurial**（2026 年的新项目用 hg 是反主流）。

14.6 国内三大平台怎么选：Gitee / GitCode / GitHub

> 同样"国内能访问的 Git 平台"，定位完全不同——**别被名字迷惑**。

维度	Gitee（码云）	GitCode	GitHub（国际）
出身	OSChina（2013）	CSDN（2023）	Microsoft（2008）
主战场	**企业 / 政企**	**个人 / 学生 / 高校**	全球开源 / 个人开发者
私有仓库	免费 5 人协作	免费	免费（无限）
访问速度	✅ 国内快	✅ 国内快	❌ 慢 / 不稳定
AI 助手	基础	**CodeGeeX + 国产 AI**	GitHub Copilot（付费）
适合"放简历作品"	较弱（社区小）	较强（CSDN 流量）	**最强**（全球雇主都看）
适合"放公司项目"	✅ 推荐	一般	❌ 涉密不能用
适合"放开源作品"	一般	一般	✅ **强烈推荐**

> **经验法则:**
>
> - **想"被看见"** → GitHub（开源）+ 顺便 Gitee / GitCode 镜像

- **公司在用** → 公司选啥你用啥
- **国内个人项目 + 偶尔不想翻墙** → Gitee 或 GitCode
- **别把鸡蛋全放国内**：万一平台挂了 / 改政策，你的东西就找不回来了

14.7 自部署 vs SaaS：什么时候自己搭一套?

场景	推荐	理由
10 人以下小团队 / 个人	**SaaS（GitHub / Gitee）**	0 运维、免费、足够用
中大型企业 / 金融 / 政府	**自部署 GitLab EE 或 Gitee 企业版**	数据可控、等保合规
想要"轻量 + 完全自主"	**Gitea 自部署**	单二进制、1G 内存就能跑
开源 / 跨地域协作	**GitHub 公有云**	网络与生态最好

> **自部署最低成本方案**：一台 1 核 2G 的云服务器 + Docker，一行命令跑起 Gitea，10 人团队完全够用。

14.8 选型决策表（一句话版）

你是谁	选
个人开发者 / 学生	**GitHub + Gitee 镜像**（必要时翻墙）
国内个人作品 / AI 时代开发者	**GitCode**（流量 + AI 辅助）
国内中小团队 / 公司项目	**Gitee 企业版** 或 **自部署 GitLab / Gitea**
金融 / 政府 / 强合规	**自部署 GitLab EE**
大型游戏 / 影视 / 汽车	**Perforce**（或主用 Git + 美术资源 Perforce）
老项目维护	看历史是啥就用啥
想"私有又不想运维"	**GitHub Enterprise Cloud**

14.9 平台之间怎么迁移

常见场景："公司在用 Gitee，想迁到 GitLab"。

```
# 1. 从 Gitee clone（bare 仓库）
git clone --bare https://gitee.com/old/repo.git

# 2. 推到新平台
cd repo.git
git push --mirror https://gitlab.com/new/repo.git

# 3. 同步所有分支 + tag + commit 历史
# （--mirror 会推送 refs/heads/*、refs/tags/*、refs/remotes/*）
```

> **坑：PR / Issue / Wiki / Release 不会自动迁移**——这些是平台数据，不在 Git 里。需要各家平台自己的"导入"功能（GitHub → GitLab 有 Import tool）。

14.10 一句话总结

> **2026 年的事实标准 = Git 工具 + GitHub 平台**。但你工作后大概率会遇到 Gitee / GitLab / 内部 Gitea——**理解它们都是"Git + 协作 UI"就够了**，剩下的只是换个 URL 和按钮位置。

---

15. monorepo 是什么：单仓多包 / 多仓多包 / 工具选型

> "monorepo" 这词在前端圈出现频率极高——Vue 3 / React / Vite / Next.js / Babel / Tauri 等"明星项目"几乎都长在 monorepo 里。但**它不是 Git 的特性，而是"仓库组织方式"的一种选择**。这一节彻底讲清"monorepo 是什么 / 解决什么 / 怎么落地"。

15.1 是什么

monorepo（monolithic repository）= 单个仓库里管理多个项目 / 包。

```
my-company/
├── apps/                          ← "应用"
│   ├── web/                       ← 用户端 Vue 3 项目
│   ├── admin/                     ← 后台管理系统
│   └── mobile/                    ← H5 / RN / 小程序
├── packages/                      ← "包"
│   ├── ui/                        ← 共享 UI 组件库
│   ├── utils/                     ← 共享工具函数
```

```
│   ├── eslint-config/          ← 共享 lint 配置
│   └── tsconfig/               ← 共享 tsconfig
├── pnpm-workspace.yaml         ← 一行配置开启 monorepo
├── package.json                ← 根 package.json
└── README.md
```

对比另一种"传统"做法：**每个项目一个独立仓库**，叫 **multi-repo**。

```
github.com/company/web          ← 一个 repo
github.com/company/admin        ← 另一个 repo
github.com/company/ui           ← 又是另一个 repo
github.com/company/utils        ← 又是另一个 repo
```

> **关键：**
>
> - **monorepo = 多个 package 在 1 个 git 仓库里；**
> - **multi-repo = 每个 package 独立一个 git 仓库。**
> - **VCS 选型（Git / SVN）和 monorepo / multi-repo 选型正交**——任一组合都可能。

15.2 解决了什么问题

痛点	monorepo 的解法
共享代码难：A 项目想用 B 项目的 `utils` → 复制粘贴 / 提私有 npm 包	在仓库内 `import '@company/utils'` 直接用，**不用发布**
版本不同步：A 项目用 vue 3.4、B 项目用 vue 3.5，升级要挨个改	一次 `pnpm update`，所有 package 同步
跨项目改一个东西：要修 8 个 PR / 8 个仓库 / 8 个 CI	1 个 PR / 1 个 CI
新人上手成本：要 clone N 个仓库、装 N 遍依赖	clone 1 个、装 1 遍
依赖图不清晰：谁依赖谁靠口头约定	`pnpm why` / `nx graph` 一键可视化

15.3 monorepo vs multi-repo 对比

维度	monorepo	multi-repo
仓库数	**1 个**	N 个
共享代码	仓库内 `import` 即可	发 npm 私有包 / git submodule
跨项目重构	一个 PR	N 个 PR 同步
CI 速度	需要**增量构建**（只跑改动的）	天然独立
权限控制	较粗（谁能 push 整个仓库）	**细**（每个 repo 独立）
仓库体积	**容易膨胀**（几个 G 起步）	各自轻量
Git 性能	巨型仓库时 `git log` 变慢	各自轻快
学习曲线	工具多（pnpm workspace / Turborepo / Nx）	**简单**
适合	紧密耦合的项目群（同一公司 / 同一产品）	完全独立的项目 / 跨组织
典型代表	Google / Meta / Microsoft / Vue 3 / React	早期 Google（pre-Piper）/ 大量小公司

> **决策一句话：**
>
> - 项目之间**共享代码 > 30%** → monorepo 划算
> - 项目之间**完全独立**（一个 2C 端、一个 2B 端、跨业务线） → multi-repo 更清爽
> - **不确定** → 从 multi-repo 起，超过 3 个项目需要共享代码再合并

15.4 monorepo 不是银弹：3 个真实坑

坑	后果	缓解
仓库体积爆炸	clone 一次 1G+，CI 拉代码 5 分钟	配 `.gitignore` 屏蔽产物；用 sparse checkout
权限粒度粗	实习生能 push 整个 monorepo	配 CODEOWNERS + GitHub 分支保护
CI 不知道"我该跑哪些"	改一行代码跑全量测试 30 分钟	引入 **Turborepo / Nx** 做增量构建

15.5 工具选型矩阵

工具	作用	适合	备注
pnpm workspace	包管理 + 软链接	几乎所有前端 monorepo	**入门首选**，见 03 第 3.5.4 节
Yarn workspaces	包管理 + 软链接	老项目 / 已用 yarn	经典方案
npm workspaces	包管理 + 软链接	想"零额外依赖"	自 npm 7 起支持，功能最简
Lerna	多包发布 + 版本管理	老牌 monorepo 工具	2022 年后已**被 Nx 收购**
Turborepo	**增量构建 + 任务编排**	中大型 monorepo	Vercel 出品，**最推荐**
Nx	增量构建 + 任务编排 + 代码生成	大型 / Angular 背景团队	功能最全、配置最重
Rush	大型 monorepo 编排	微软系（Office / Teams）	适合 100+ 包
Bun workspaces	包管理 + 软链接	想"用 bun 一把梭"	bun 内置，比 pnpm 更快但生态新

15.6 pnpm workspace 最小落地：4 步走起来

> 承接 03 第 3.5.4 节。这里给一个完整的"从 0 到跑起来"流程。

```
# 1. 准备目录结构
mkdir my-monorepo && cd my-monorepo
mkdir -p apps/web packages/ui

# 2. 根 pnpm-workspace.yaml
cat > pnpm-workspace.yaml <<'EOF'
packages:
  - 'apps/*'
  - 'packages/*'
EOF

# 3. 根 package.json
cat > package.json <<'EOF'
{
  "name": "my-monorepo",
  "private": true,
```

```
  "scripts": {
    "dev": "pnpm -r --parallel dev"
  }
}
EOF

# 4. 在 packages/ui 里写一个共享组件（具体略）
# 然后在 apps/web/package.json 引用
# { "dependencies": { "ui": "workspace:*" } }
```

> 装一次，跑全部：

```
pnpm install              # 装所有
pnpm --filter web dev     # 只跑 web
pnpm -r --parallel dev    # 全部并行 dev
```

15.7 Turborepo：把"增量构建"做到极致

> 假设 monorepo 有 50 个 package，改了 1 个文件——**理想情况是只跑这 1 个文件相关的测试**。这就是 Turborepo 干的事。

```
# 装 Turborepo
pnpm add -D turbo
```

`turbo.json`（项目根目录）：

```
{
  "$schema": "https://turbo.build/schema.json",
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**"]
    },
    "test": {
      "dependsOn": ["^build"],
      "outputs": ["coverage/**"]
    },
    "lint": {},
    "dev": { "cache": false }
  }
}
```

```
# 自动找"哪些包要重跑"
turbo run build
```

```
# 看任务图
turbo run build --graph
```

> 实战收益：CI 时间从 30 分钟 → **3 分钟**。

15.8 Nx：更重、更全、更适合大企业

维度	Turborepo	Nx
增量构建	✅ 极简	✅ 功能更多
受影响分析	手动写 `dependsOn`	**自动推断**
代码生成	无	✅ `nx g component xxx`
远程缓存	Vercel 平台	Nx Cloud
插件生态	少	丰富（React / Next / Node / Express …）
学习曲线	低	中
适合	5–50 个 package	50+ 个 package / 大型 monorepo

> 选哪个？
>
> - **5–20 个包 + 中小团队** → Turborepo
> - **20+ 个包 + 复杂依赖图 + 大公司** → Nx
> - **啥也不想管** → pnpm workspace 裸跑（够用就好）

15.9 monorepo × Git 平台：坑与解

坑	表现	解决
GitHub 仓库体积警告	单仓 > 5 GB 时发邮件	sparse checkout / Git LFS / 拆仓库
PR 改动太大	monorepo 一个 PR 改 30 个 package	**限制范围**（用 changeset / nx affected）
CODEOWNERS 粒度	一人 review 整个 monorepo	`.github/CODEOWNERS` 按路径分 owner
CI 拉仓库慢	每个 job 都要 clone 整个 monorepo	用 Turborepo 远程缓存 / 只拉需要的子目录

坑	表现	解决
package 之间循环依赖	A 依赖 B、B 又依赖 A	工具检测（`madge` / `nx graph`）

15.10 真实世界：哪些项目在用 monorepo

项目	monorepo 工具	规模
Vue 3	pnpm workspace	~30 个 package
React	Yarn workspaces	~50 个 package
Vite	pnpm workspace	~20 个 package
Next.js	Turborepo（迁移后）	数百个 package
Babel	Yarn workspaces + Lerna	100+ 个 package
Google	自研 Piper	**几十亿行代码**
Meta（Facebook）	自研	巨型
Microsoft	部分 monorepo + Rush	Office / Teams 等

> 看到没？**全世界最顶尖的开源 / 大厂项目 80% 在用 monorepo**——它不是"小公司玩具"，而是**应对复杂项目的标准答案**。

15.11 一句话总结

> **monorepo = 把"多个相关项目"放一个 Git 仓库里，配 pnpm workspace + Turborepo 落地**。它解决"共享代码 / 同步升级 / 跨项目重构"三大痛点；门槛是工具变多、CI 要做增量。如果你项目数 ≥ 3、共享代码 ≥ 30%，**就该上 monorepo**。

---

13. 小结与自检

> 想对比 GitHub / GitLab / Gitee / SVN 等 9 个平台、以及搞清楚 monorepo 是什么，见 第 14 章 和 第 15 章。

13.1 自检清单

#	题目	关键点
1	解释工作区 / 暂存区 / 版本库三个区	三区转换图
2	`git reset` / `git revert` 区别	reset 改历史、revert 加反向 commit
3	在 GitHub 创建一个 repo，本地 `git push` 推上去	全流程
4	拉一个 feature 分支、写代码、提 PR、合并	完整 PR 流程
5	模拟一个冲突并解决	`<<<<<<<` 标记
6	写一个 Conventional Commit	`feat(scope):` 格式
7	配 SSH key 并 `ssh -T git@github.com` 验证	免密推送
8	用 `git reflog` 找回误删分支	reflog 是后悔药
9	在仓库里加 GitHub Actions：push 时跑 `pnpm test`	CI 入门
10	写一个 `.github/ISSUE_TEMPLATE/bug.md`	Issue 模板

13.2 推荐练习

1. 把本教程的 README 仓库 fork 到自己账号，clone 下来，试着提一个 PR 改个错别字（**真实 PR 流程体验一次就忘不了**）；
2. 在 GitHub 上开一个私人项目，模拟"主分支 → feature 分支 → PR → 合并 → 删除分支"的完整流程；
3. 给一个项目配 GitHub Actions：每次 push 自动跑 `pnpm lint && pnpm test && pnpm build`；
4. 故意制造一次合并冲突，体验完整解决过程。

13.3 必备速查表

想知道	跑这个
当前状态	`git status`
历史（简洁）	`git log --oneline --graph`

想知道	跑这个
历史（详细）	`git log --stat`
某文件改了什么	`git diff path/to/file`
找包含某关键字的 commit	`git log -S "keyword"`
找某行是谁改的	`git blame path/to/file`
看某次 commit 内容	`git show <hash>`
远程地址	`git remote -v`
改 commit 信息（最近一次）	`git commit --amend`
救回误删	`git reflog`

推荐阅读

- Pro Git 中文版：https://git-scm.com/book/zh/v2
- Learn Git Branching（交互练习）：https://learngitbranching.js.org/?locale=zh_CN
- Conventional Commits：https://www.conventionalcommits.org/zh-hans/
- GitHub Actions 官方文档：https://docs.github.com/zh/actions
- Oh My Git（游戏化学习）：http://ohanigit.com/